

Google cluster data analysis with Spark

Marian Aldescu

Mahdis Ghaffarian

Master M2 – Université Grenoble Alpes — June 5, 2026

For this project, we used mainly **im2ag-mandelbrot** computer with the following specs:

- Software: Debian GNU/Linux 8, python3.4, spark-3.0.1
- Hardware: Common KVM, 2.5GHz, 8 cores, 1 thread/core, 32GB RAM

1 Questions

For Q1-6, we processed all the available files: `machine_events`, `task_events`, `job_events`. Considering that `task_usage` files are very large, compared to the previously mentioned, for Q7-9 we chose to process 50 files, a number which still allows us to extract sufficient information in order to provide a proper answer to questions.

Due to memory constraints, processing the files all at once is unsuitable, hence a solution that we propose is to compute chunks of **N** files(e.g. for `task_events`, `job_events` $N=100$, for `task_usage` $N=4$ or 10), appending the partial processed information to a global RDD accumulator, and then, we continue the analysis on it.

Our implementation is done using RDDs, and based mainly on lazy evaluation. The actual computation is done normally after accumulating data from **N** files in the current RDD.

Q1. A. *What is the distribution of the machines according to their CPU capacity?*

Answer:

In order to extract the CPU capacity, we use the `machine_events` file. Since there can be multiple events with the same `machine_ID`, we filtered the entries to have a list with no `machine.ID` duplicates, then we counted how many machines correspond to each CPU type. Check Table 1 for results. Notice that most of the machines contain half of the maximum CPU capacity.

CPU	Machines
0.25	126
0.5	11659
1	798

Table 1: Machines distribution over CPU capacity

B. *What is the distribution of the machines according to their RAM capacity?*

Answer:

We proceed in the same manner as for the previous task. In Table 2, the final results. It's interesting to notice that the RAM distribution is fragmented in more types than the CPU distribution. The largest number of machines for a category(6732) corresponds to half of the maximum RAM capacity. At the same time, it seems that the number of machines that contain the maximum RAM capacity(795), could be the same that contain the maximum CPU capacity. We will check this in the following question.

RAM	Machines
0.1241	54
0.2493	3866
0.2498	126
0.4995	6732
0.5	3
0.749	1003
1	795

Table 2: Machines distribution over RAM capacity

C. *What is the distribution of machines as a function of CPU and RAM capacity?*

Answer:

Continuing the previous questions, we want to see the distribution of machines for each pair (CPU, RAM) quantity, idea taken from [2]. We notice in Figure 1 that the majority of machines have medium resources, with a slightly larger variation in the RAM axis. Our previous intuition that there exists a subgroup of machines with a maximum amount of resources is now confirmed(the pink circle).

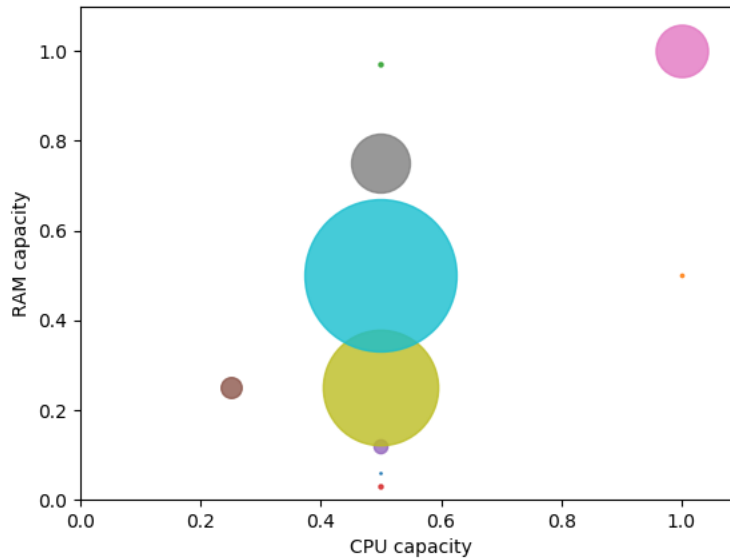


Figure 1: Distribution of machines over CPU and RAM. The diameter of the circles is proportional to the number of machines in a category.

Q2. *What is the percentage of computational power lost due to maintenance (a machine went offline and reconnected later)?*

Answer:

For a machine, we will consider the lost computational power as the time interval in which the machine is down('DOWN' time): after an event 'Remove'. we will compute the total time during a machine is running('UP' time) and from this we can easily obtain the total 'DOWN' time.

During the trace, a machine is usually passing through a succession of alternate operations 'Add' and 'Remove'('Update' operations do not change anything), therefore the total up can be computed as a sum of $\Delta t_n = time_R - time_A$, where $time_R$, $time_A$ are successive timestamps of the same machine(A=Add, R=Remove).

$$totalTime_{UP} = time_{R1} - time_{A1} + time_{R2} - time_{A2} + \dots \quad (1)$$

To compute $totalTime_{UP}$, we use first a **map** operation to associate a timestamp for each machine, and the timestamp is negative if the event has the type 'Add'. Then, with a **reduceByKey** we compute the sum from Eq.(1) for each machine_ID.

An important detail is that the last event for each machine has indefinite action time, hence we will consider the stop trace time, as the timestamp of the last event from the dataset.

After running the computations, we obtained the following percentages:

Computation time $\approx 96.1\%$

Lost time due to maintenace $\approx 3.9\%$

Q3. *On average, how many tasks compose a job?*

Answer:

We use the *task_events* files to extract the pairs of (job_ID, task_index) from each event. Because for the same task_index there can be multiple entries in the files, we want to keep an unique occurrence for each one of them, so we remove any duplicates using a **distinct** operation. We want then to count all the jobs, which can be done by applying a **map**(job→(job, 1)), then **reduceByKey** with sum and **count**. The number of tasks is furthermore obtained using **map** and **reduceByKey** in a similar manner. In total, we obtained:

#Jobs: 672004

#Tasks: 25424731

Average: 37.8341959274 tasks/job

Q4. *What can you say about the relation between the scheduling class of a job, the scheduling class of its tasks, and their priority?*

Answer:

From the *job_events* files we extract all the unique occurrences of the pairs (job_ID, job_class). This means to apply the **distinct** operation at each time a file is appended to the accumulator RDD. From the *task_events* files we extract the tuples with the following fields: (job_ID, task_index, task_class, task_priority). Also for these, we keep only the unique entries. Since

the task_index was used to ensure the uniqueness of each task, we can remove it from the tuple.

The purpose is to associate to each (job_ID, job_class) pair its corresponding (job_ID, task_class, task_priority) entry from the task_events, hence we use a **join** to combine them, then remove the job_ID, it was useful only to merge the data. Now that we have all the tuples (job_class, task_class, task_priority), we can easily compute the average task_class and task_priority with a **reduceByKey** operation (to calculate the sum) and **count**, merge the 2 results with **join**, where the key is the job_class and compute the division between the sums and counts with **map**.

The results can be seen in Figure 2. It can be noticed that all the tasks inside a job have the same class as the job (Figure 2a). Regarding the average task priorities/job class (Figure 2b), we can infer that usually, the jobs with the classes 0 and 2 contain tasks with low priorities of average ≈ 3.5 . On the other hand, the job class 1 contains higher priorities tasks, and more, the class 3 has the largest task priority average, of 5.85.

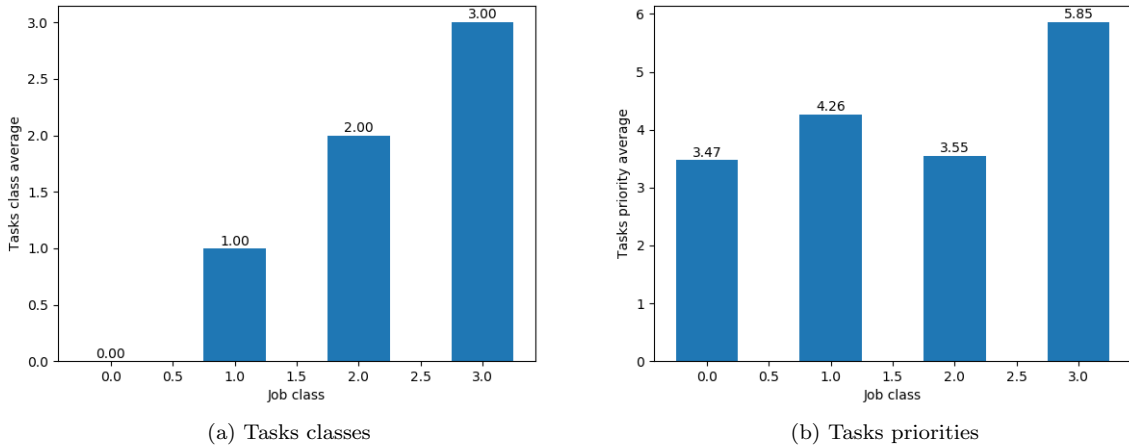


Figure 2: Average tasks classes and average tasks priorities for each job class

Q5. *Do tasks with low priority have a higher probability of being evicted?*

Answer:

The intuition for this question is to count how many tasks are evicted for each priority, but it is not sufficient, because there can exist certain priorities which contain a lot more tasks than others, thus, we want to average the number of evicted tasks by the total number of tasks having the same priority.

Note that according to the Google documentation, tasks can be evicted multiple times, thus, in our measure we intend to count such cases each time they appear in the events files. We then count how many tasks exist in total for each priority. The number of evicted tasks is obtained by first filtering out the non-EVICT tasks, then count them with **countByKey**, **join** the result with the total tasks/priority and compute the division of evicted tasks/total #tasks.

Figure 3 confirms us that the jobs with lower priority are more likely to be evicted, especially those with priority 1. Surprising is that also the tasks with very high priority (e.g. 9 and

10) are prone to eviction. We don't have a precise explanation for this, but a possible cause is that at a certain moment, a lot of high priority tasks are running, so the competition for resources results in the eviction of some of them. Another reason could be the existence of several greedy tasks, that request many resources, so that in the end, they will be ejected.

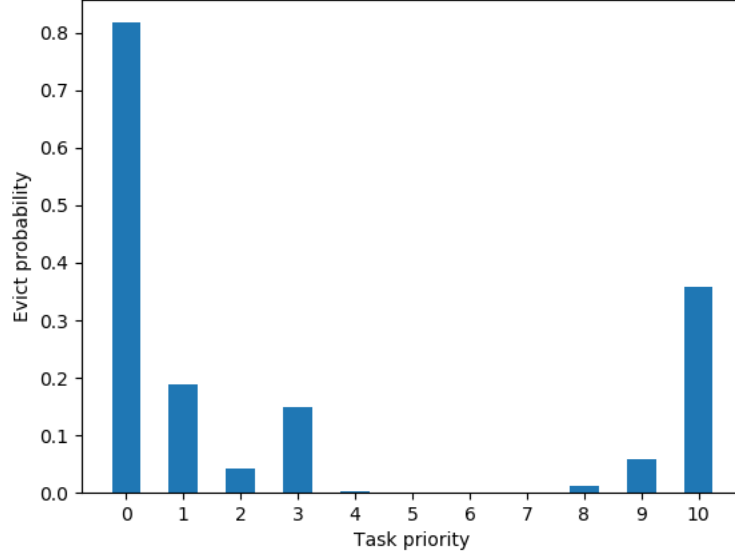


Figure 3: Eviction probability for each task priority

Q6. *In general, do tasks from the same job run on the same machine?*

Answer:

From the task_events files we extract the pairs (job_ID, machine_ID) pairs and remove the duplicates with **distinct**. We don't need to keep any other information about the tasks (like task_index), because for example if 2 tasks of the same job run on different machines, these will reflect in 2 different pairs (job_ID, machine_ID). We can then easily compute on how many machines each job is running with **reduceByKey** (where the job_ID is the key), obtaining pairs (job_ID, #machines), **map** these to (#machines, 1) so that apply again **reduceByKey** to obtain the number of jobs whose tasks run on a certain number of different machines.

Figure 4a shows that usually the jobs are situated in one of the two extremities: 1st in which many jobs run on a small number of machines, 2nd where several jobs are running on many machines. After counting how many jobs are running on more than one machine, there are:

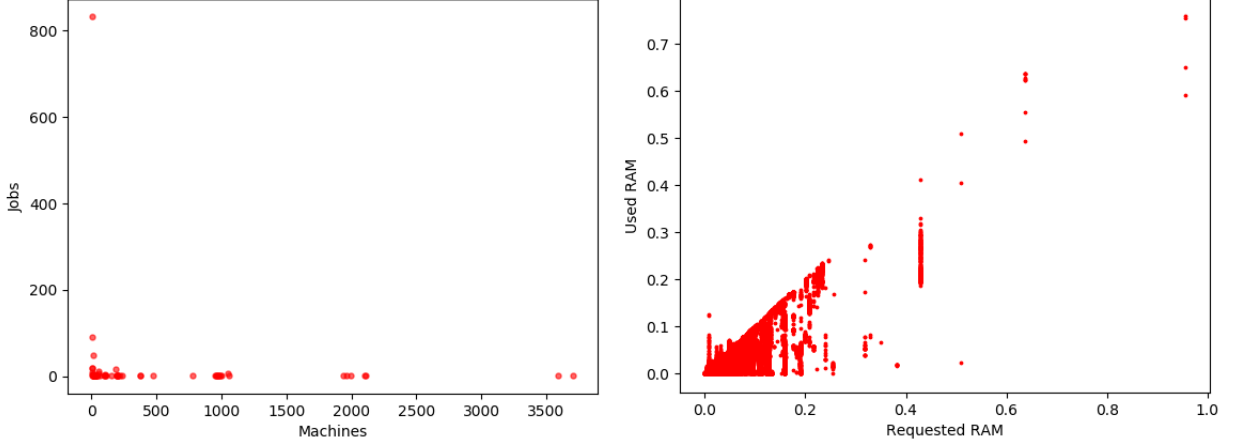
- 323 jobs running on more than one machine
- 832 jobs running on a single machine

We can, therefore, conclude that normally jobs run on the same machine.

Q7. *Are the tasks that request the more resources the one that consume the more resources?*

Answer:

We propose to analyse the quantity of RAM requested vs the quantity of RAM used. From the task_events files we extract tuples of (job_ID, task_index, event_type, req_RAM) and



(a) Plot showing how many jobs are running on a certain number of machines. (b) The amount of RAM requested by each job vs the actual used RAM

Figure 4

remove the duplicates, then filter only those that contain the event_type SCHEDULE. From the task_usage files we keep similar pairs of (job_ID, task_index, used_RAM) and remove the duplicates as usual. With **reduceByKey** we sum up all the used RAM by a task, where the key is the pair (job_ID, task_index) and divide these quantities to the number of samples computed with **countByKey**.

In Figure 4b, we can visualize the final result. Two interesting are observed: some tasks use approximately the amount of RAM that they request, hence the red points are situated on the diagonal of the plot. The second one is that many tasks use less memory than they actually requested. These being said, it is difficult to offer a straight answer to the question. It is clear that there is a lot of space to improve the allocation of memory to the tasks, as many of them use less than requested, but this can not be generalized, as there is a considerable number of tasks that respect their initial demand.

Q8. *Is there a relation between the amount of resource consumed by tasks and their priority?*

Answer:

We try to answer to this question by associating for each task priority the average memory consumed in a certain time interval. We extract from the task_events files the tuples (job_ID, task_index, priority) and remove the duplicates. From the task_usage files we extract the tuples ((job_ID, task_index), used_RAM) and we keep the unique entries. Considering that for a task, the memory usage sampling is done periodically, we intend to compute an average of those values, therefore we apply **reduceByKey** with sum, compute then separately the #apparitions of each key (job_ID, task_index), and **join** the sums and counts in an unique RDD and compute the average with **map**. To associate the average memory consumed with the priorities, we **join** them and we compute the average RAM used, but for the tasks that have a certain priority.

Figure 5 shows the results. The tasks with high priority also consume more resources, that was almost expected to happen.

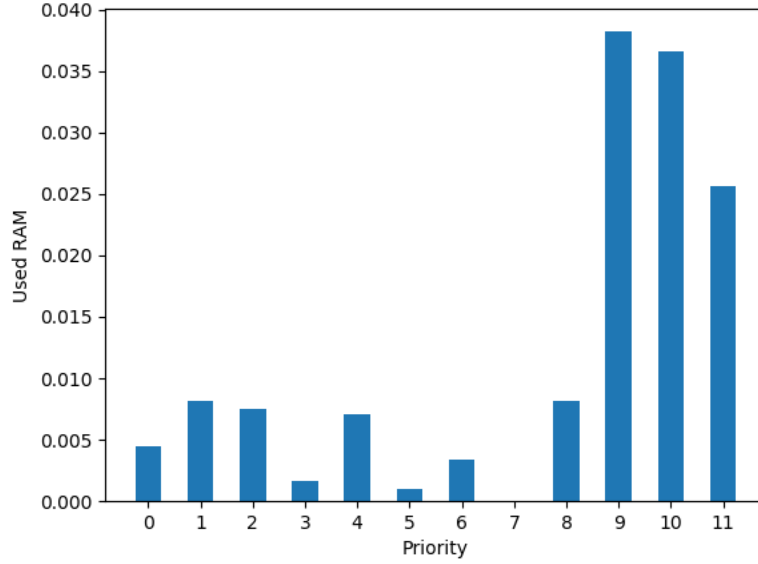


Figure 5: Used RAM average for each job priority

Q9. *Can we observe correlations between peaks of high resource consumption on some machines and task eviction events?*

Answer:

We want to plot the evolution of the evicted tasks/time interval(60min for example) for several days, and also the evolution of the average memory reserved by the system in the same period. From the task_events files we filter only the pairs (timestamp, event_type) that have the type EVICT and the count how many tasks are evicted in a frame of time with **reduceByKey**. For the task_usage files, we sum the used RAM quantities for each time interval of 60min.

In order to observe such correlations, we selected a time interval of $\approx 90h$, starting after 80h from the beginning of the trace. In Figure 6, we notice the increasing memory usage for the first $\approx 40h$, that results in 3 massive tasks eviction phases, represented by the first spikes in Figure 7.

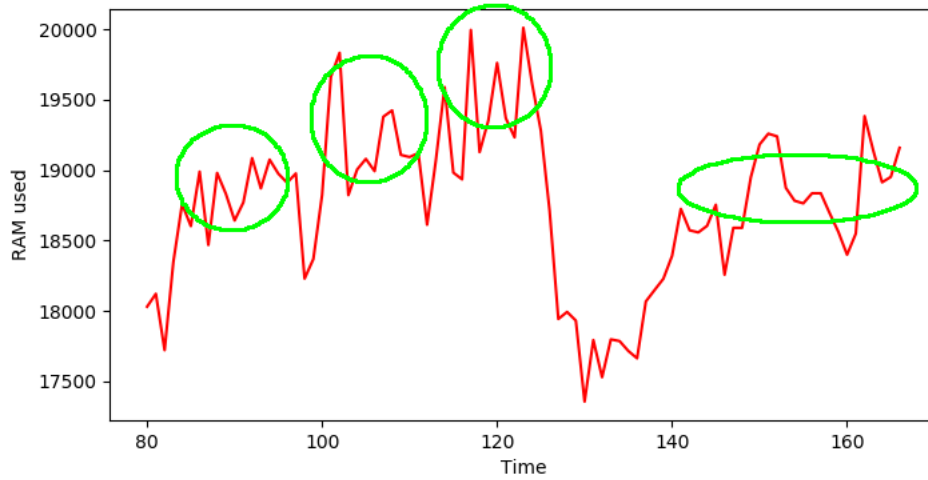


Figure 6: Used RAM evolution in time(h)

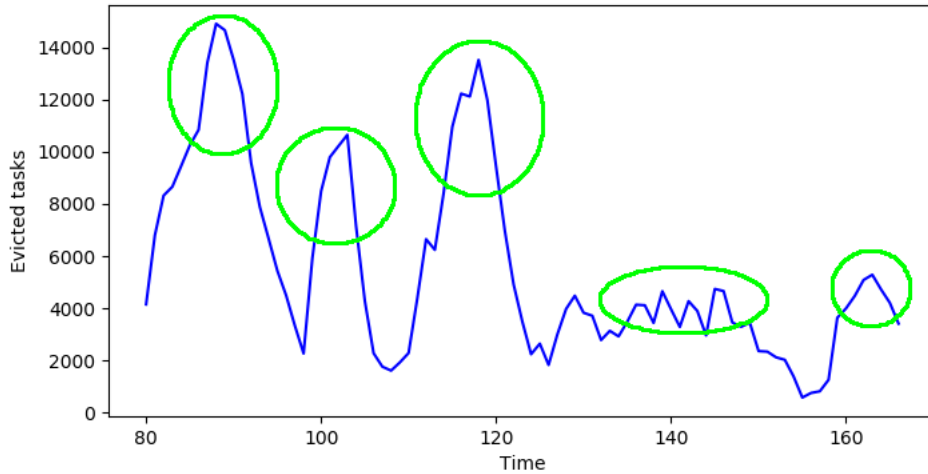


Figure 7: # evicted tasks in time(h)

2 Performance

- Application level

In this part we have applies two ways to compare the performance in the application level.

First, as we know using same RDD multiple times during computation could gets a lot of time; so we could cache the first RDD results in the memory in order to save it and use it for the rest computation and not to recalculate it again each time.

In Table 3 it can be seen two different results that we got for question 7 as an example(commented out in the code).

Second, as RDDs have lazy evaluation so having more actions on RDD could impact on the performance; therefore as an example we have applied more actions(e.g collect())

Question	Without caching	With cache
q7	2560.33	2371.83

Table 3: Performance comparison with and without cache for Q7

on question six to compare both results in terms of time we could get, and as it can be seen in Table 4, we have a better performance when we are not applying extra actions on RDDs(commented out in the code).

Question	Lazy evaluation	No lazy evaluation
q6	30.33s	787.56s

Table 4: Performance comparison using or not lazy evaluation for Q6

- System level:

In this part we tried to set some system configuration for Memory and CPU as below:

```
param_nb_threads = 'local[{}]' .format(nb_threads)
sc_conf = SparkConf()

sc_conf.set('spark.executor.memory', '32G')
sc_conf.set('spark.driver.memory', '32G')
sc_conf.set('spark.driver.maxResultSize', '32G')
```

In this configuration we can set the number of cores, so we choose to compare the computation for 4 and 8 cores respectively. The results can be seen in we got the below results which can be seen that mostly by 8 cores we have a better performance as we expected.

We also mention that we have found out that on Mandelbrot the resources are shared, hence that could be a reason which affects negatively the performance(Table 5).

Question	q1	q2	q3	q4	q5	q6	q7	q8	q9
Mandelbrot_Time_4cores(s)	2.93	2.69	694.07	1267.64	885.96	28.59	2841.72	2960.43	292.21
Mandelbrot_Time_8cores(s)	2.50	2.95	397.63	908.60	552.64	30.33	2560.33	2345.13	214.21

Table 5: Performance comparison between 4 and 8 cores on Mandelbrot

3 Review of the publication from 2020 entitled "Borg: the Next Generation"

The fresh cluster trace, published in 2019[1], contains some interesting differences, compared to the one from 2011. A first aspect to notice is the greater variety of the resources(CPU and memory) in the new trace, compared to a more compact distribution in the older one. We refer to Figure 1, for this. Notice how the majority of the machines were located in the medium resources area of the plot, whereas now, they show a greater variety of the CPU and memory organization.

In 2011, the data set is composed of information of a single cell, in which the resources are directed mainly to the production tier. In 2019, the new trace shows an interesting change of the usage proportions, this time for 8 cells. First, we notice the apparition of a new tier(Mid-tier). Second, from cell to cell, the overall workload is unequally distributed, it looks like some cells are preferentially assigned jobs from specific tiers. Therefore, we can talk about a specialization of the cells, to work more on tasks that have certain priorities. Also, beb tier gained a higher proportion in all the clusters, compared to 2011.

Even though the scheduling rate is today higher, both in terms of jobs(x3.7) and tasks(x7), which does not surprise us, the number of machines in a cluster remains almost constant. It seems to be preferable an increasing on horizontal(# of clusters) instead of adding more machines to the cluster.

The new trace comes together with a new mechanism of dynamically adjustment of the task allocated resources(Vertical scaling), called Autopilot, that we consider it an essential starting point of reducing the difference between the amount of requested and used resources, hence, directly resulting in cost savings. We believe that analyzing these old traces could help perform better predictions when allocating resources in the future, with the mentioning that this is not a trivial task, due to the huge quantity of data generated during monitoring.

2 important improvements worth mentioning in the 2019 trace: the machine utilization has increased for both memory(3-30%) and CPU(20-40%), but at the same time reducing the variation. This shows that high qualitative schedulers have a crucial impact on the equal distribution of resources and stability of the usage in time. Regarding this, the existence of multiple schedulers and of the *batch scheduler* shows to be effective. It seems that the existence of some kind of a hierarchy (batch scheduler that pass a job to the regular Borg scheduler) achieves a better control of the allocation of cluster resources.

4 Deploying in the Cloud

In order to deploy on cloud and compare our results from Mandelbrot with a virtual machine in the Cloud, we simply create a VM and provide it with Terraform.

The type of VM that we created is "n2-highmem-8" in the region "europe-north1". This machine has 8 vCPUs, 64 GB memory. We also needed to resize the disk space as it is only 10 GB at the beginning.

Results by considering 4 cores:

Question	q1	q2	q3	q4	q5	q6	q7	q8	q9
Mandelbrot_Time(s)	2.930	2.693	694.076	1267.641	885.961	28.592	2841.720	2960.433	292.219
Cloud_Time(s)	1.924	2.355	665.802	1199.499	870.775	20.971	2539.796	2522.166	245.410

Table 6: Performance comparison for 4 cores between Mandelbrot vs Google Cloud

Results by considering 8 cores:

Both servers get us more and less the same results as they were tested in the same configurations in terms of number of cores and Memory; but still there are some differences which show us reaching a better performance does not only depends on CPU and RAM, we need to consider other aspects like the type of storage system and also the main architecture that

Question	q1	q2	q3	q4	q5	q6	q7	q8	q9
Mandelbrot_Time(s)	2.506	2.952	397.635	908.607	552.645	30.337	2560.334	2345.135	214.215
Cloud_Time(s)	1.632	2.111	464.452	1054.545	672.250	20.749	2344.001	2279.165	220.513

Table 7: Performance comparison for 8 cores between Mandelbrot vs Google Cloud

is used for CPU and RAM. That is why by using 4 cores(Table 6) we have a better results (less time) in the Cloud than Mandelbrot while considering 8 cores(Table 7) gave us more time in the Cloud.

At least, as we expected, in both servers we have a better performance when using a higher number of CPUs(8 vs 4).

References

- [1] Joseph Hellerstein Charles Reiss, John Wilkes. Google cluster-usage traces: format + schema.
- [2] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. Borg: The next generation. In *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys '20, New York, NY, USA, 2020. Association for Computing Machinery.